

Seminar Presentation

ARM 환경에서 FAEST 서명 가속화 연구

Accelerating FAEST Signatures on ARM:
NEON SIMD AES and Parallel VOLE Optimization

발표자 이승원

일시 2026년 3월

YOUTUBE <https://youtu.be/pXZ9pe65XUA>



기술적 특징 및 핵심 개념



AES 기반 안전성 (OWF)

검증된 대칭키 암호인 AES를 단방향 함수(One-Way Function)로 사용하여 보안성 입증에 용이함



VOLE-in-the-Head 구조

MPC-in-the-Head 패러다임을 발전시킨 형태로, VOLE을 사용하여 영지식 증명을 효율적으로 구성

1. MPC: 서로 신뢰하지 않는 참여자들이 비밀 데이터를 공개하지 않으면서
공통의 계산 결과만 얻는 프레임 워크

2.VOLE: 증명자와 검증자가 유한체 위에서 선형 관계를 만족하는 비밀 값을 나누어 갖는 프로토콜

3.in-the-Head: 1명의 증명자가 가상의 참여자를 로컬메모리에서 시뮬레이션 하는 패러다임



작은 서명 크기 (Signature Size)

FAEST-128s 기준 약 4.5KB로, SPHINCS+(7.8KB) 대비 42% 작아 전송 효율이 높음

⚠ Trade-off: 서명 크기는 매우 작으나, 증명 생성을 위해 수천 번의 AES-CTR(PRG) 연산이 필요하여
계산 비용이 높음



알고리즘 흐름

1

KeyGen: 비밀키 k 선택, $y = Ek(x)$ 계산 → 공개키 (x, y)

2

Sign: VOLE 생성

3

Verify: 시드/VOLE 재구성 후 제약 조건 검증

VOLE 핵심 관계식

$$q = u \cdot \Delta + v$$

(u : 증명자 벡터, Δ : 검증자 챌린지, v, q : 공유 값)

변형 버전 비교 (Variants)

버전	특징	서명 크기
FAEST	표준 AES 키 스케줄 사용	표준
FAEST-EM	Even-Mansour 구조 (키 스케줄 생략)	더 작음 (약 3.9KB)



KeyGen

🎲 Pick random AES key k

📄 Compute $y = Ek(x)$

Output
 $pk=(x, y), sk=k$

Sign (VOLE-in-the-Head)

🌲 GGM Tree Seed Expansion

🔗 PRG (AES-CTR) → Random Vectors

🔗 Compute VOLE Correlations

✓ ZK Proof (AES Circuit Integrity)

Fiat-Shamir Transform

Verify

📁 Reconstruct Seeds (Open Path)

♻️ Reconstruct VOLE & q

🔍 Check Constraints & Consistency

Result
Accept / Reject

✓ 핵심 수식: VOLE 상관관계

$$q = u \cdot \Delta + v$$

u, v : Prover's Vectors Δ : Global Challenge q : Verifier's Output

⚙️ 성능 핵심: PRG 구성



AES-CTR Mode



- ✓ 독립적 블록 (병렬화 가능)
- ✓ VOLE 생성 시 수천 회 호출



Register-resident S-box

256바이트 크기의 S-box 전체를 16개의 NEON 레지스터(v16~v31)에 상주시켜 유지.

→ 메모리 로드(LD) 연산 제거 및 캐시 접근 패턴 은닉

4-Stage TBL/TBX SubBytes

단일 TBL 명령어는 최대 64바이트 테이블만 참조 가능.

이를 극복하기 위해 TBL + 3개의 TBX 명령어를 체이닝하여 256바이트 전체 범위 조회 수행.

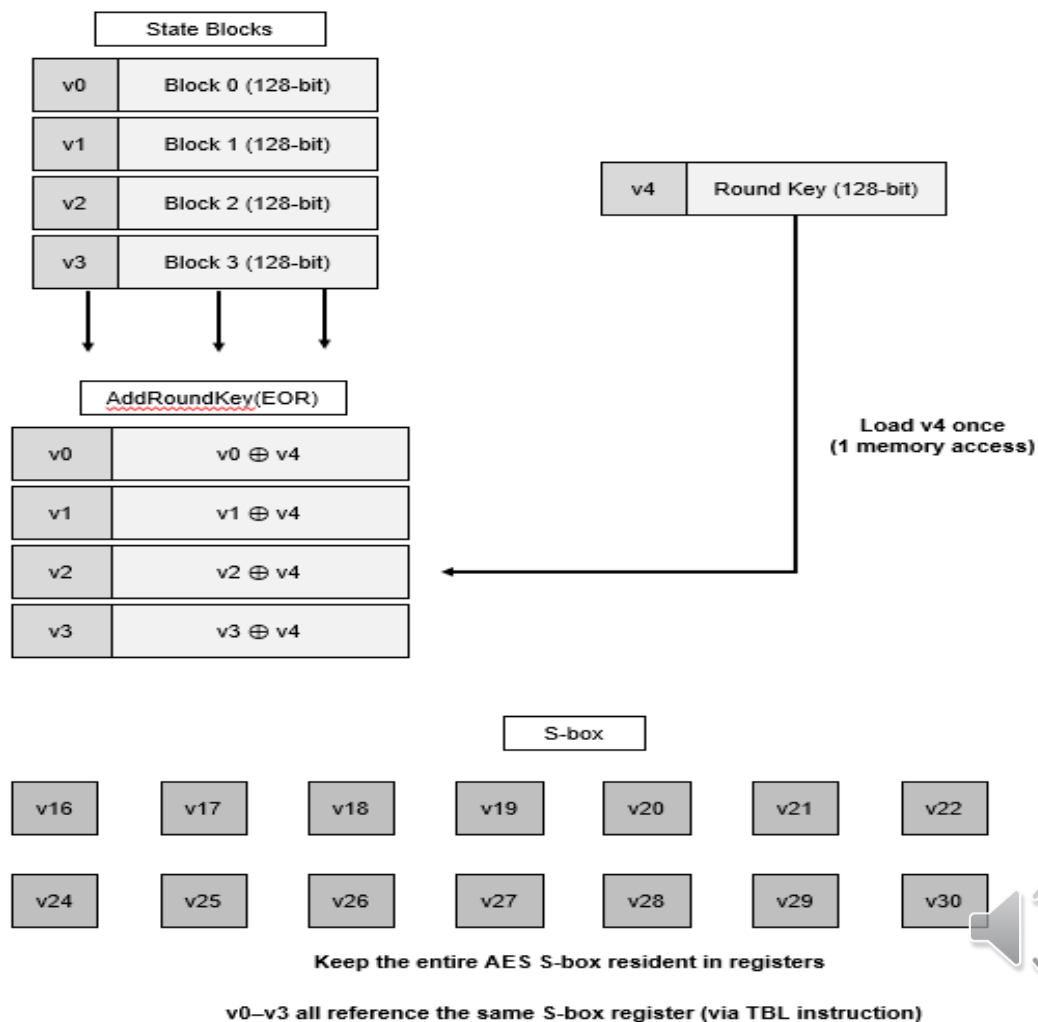
✓Branch-less Constant Time Operation

4-way / 8-way 병렬 처리

여러 블록을 동시에 암호화하여 파이프라인 효율 극대화.

- 4-way: 4개 상태 블록 + S-box 상주
- 8-way: 8개 상태 블록 + S-box + 상수/임시

NEON Register Allocation (8-way Parallel)



Fixed-length PRG (Loop Unrolling)

FAEST 트리 확장은 항상 2λ 또는 4λ의 고정된 출력을 요구함.
→ 동적 루프 제거 및 언롤링(Unrolling) 적용으로 분기 오버헤드를 원천 차단.

8-way IV 일괄 생성 (Batch IV)

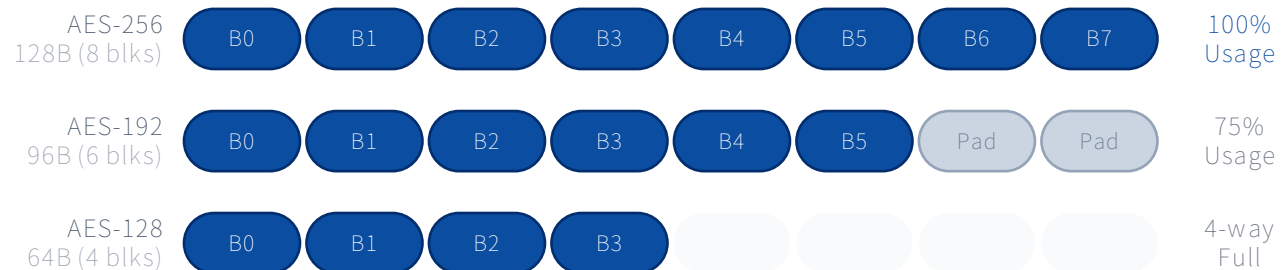
CTR 모드의 카운터(IV) 8개를 메모리 버퍼에 미리 연속 생성.
단일 AES_8way 호출로 8블록(128바이트)을 동시에 암호화하여 처리량 극대화.

계층적 처리 (Hierarchical Strategy)

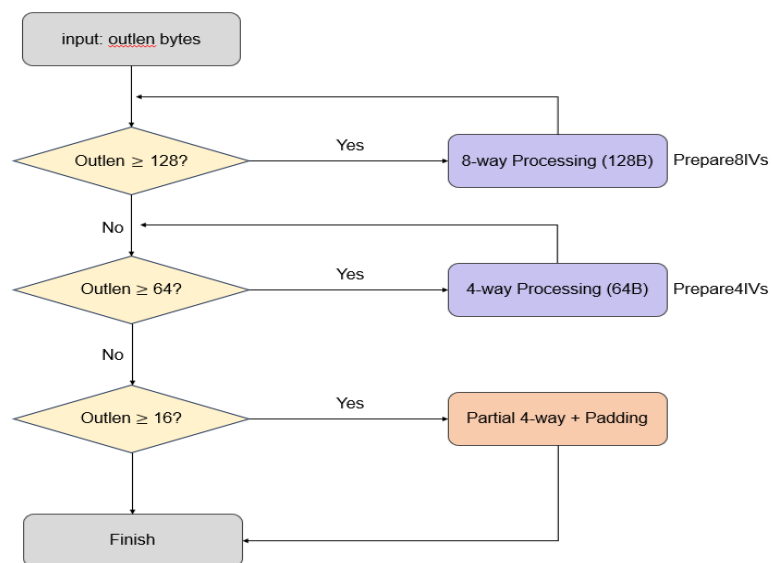
남은 데이터 양에 따라 최적의 병렬 폭을 자동 선택.

- $\geq 128B$: 8-way (최우선)
- $\geq 64B$: 4-way
- 나머지: 4-way 부분 슬롯 + 제로 패딩

Security Level별 PRG 슬롯 활용 (4λ 출력 기준)



Hierarchical Strategy



07 제안 기법 3: pthread 기반 VOLE 병렬화

오프셋 사전계산 (Offset Precomputation)

기존 구현의 순차적 의존성을 제거하기 위해, 파라미터 기반으로 각 라운드의 Seed와 Output 메모리 위치를 미리 계산.
→ 앞선 라운드 완료 대기 없이 즉시 병렬 쓰기 가능

배치 단위 스케줄링 (Batch Scheduling)

전체 VOLE 인스턴스를 가용 코어 수에 맞춰 배치(Batch) 단위로 분할 실행.

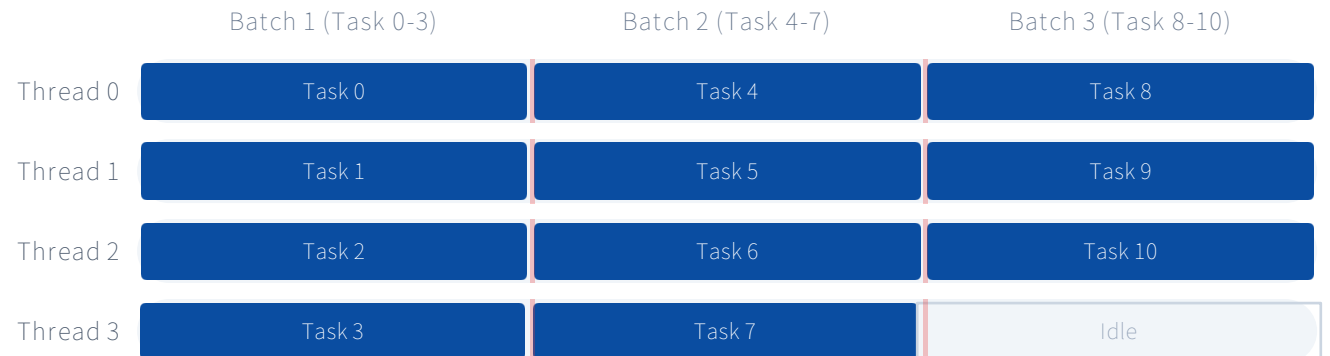
- `pthread\_join`을 동기화 장벽(Barrier)으로 활용
- 스레드 생성/해제 오버헤드 및 컨텍스트 스위칭 최소화

데이터 레이스(Data Race) 원천 차단

복잡한 Lock/Mutex 없이 설계 레벨에서 충돌 방지.

1. Read-Only: IV, 파라미터 등 공용 데이터
2. Independent Write: 사전에 분리된 비중복 출력 버퍼

Batch Execution Timeline



↔ Synchronization Barrier (pthread_join)





Problem

VOLE 단계에서의 PRG(AES-CTR) 병목현상 발생.
ARM 환경에서 Crypto Extensions 미지원 시 심각한 성능 저하 및 전용 최적화 부재.



Solution

1. NEON SIMD AES (Register-resident S-box)
2. PRG/OWF Vectorization (4-way/8-way)
3. Parallel VOLE (pthread batching)



Result

RPi4: ×136.9 / M2: ×330.1
순수 SW AES 대비 획기적 성능 향상.
ARM 전반에서 일관된 가속 효과 입증.

T h a n k Y o u

